ASHESI UNIVERSITY

ASHESI UNIVERSITY COLLEGE

APPLIED PROGRAMMING PROJECT

APPLIED PROGRAMMING FOR ENGINEERS

MAY 30 2026

RLC CIRCUIT DYNAMICS

MR. PATRICK DWOMFUOR

PROJECT MEMBERS: Cal Afun, Darius Dewortor, Jachin Kpogli, Nii Dodoo, Deo Abban

Table of Contents

Introduction

A lot of electronic devices around us, from a simple radio to the most complicated communication system you could think of, share one subtle fundamental characteristic: they do not respond instantaneously to external stimuli (Ayankop-Andi, Umar & Ibeh, 2017). Let's take a radio, for example: when a radio is turned on across different frequencies, the circuit does not immediately lock onto a single station. Instead, it fluctuates briefly before settling on one frequency. These small delays, which are common in many of our gadgets, are actually intentional and not a result of inefficiency in these devices. *But what exactly happens when we take away these delays?* Contrary to what you may think, taking away these delays would result in a high level of instability. For this to be more relatable, let us take a camera flash circuit. When the button is pressed, the capacitor charges slowly and then discharges rapidly through a circuit. If the energy were released instantly rather than being stored in a capacitor, you would get a destructive current spike, which would burn out the flash tube immediately.

What we are observing is not inefficiency but a controlled exchange of energy between three fundamental elements: resistance, inductance, and capacitance. The capacitor stores energy in an electric field, the inductor stores energy in a magnetic field, and the resistor regulates how quickly the energy is dissipated. Together they determine how a system responds when it is disturbed: whether it settles smoothly, oscillates before stabilising, or becomes unstable. This is the basis of the RLC circuit

Problem Definition

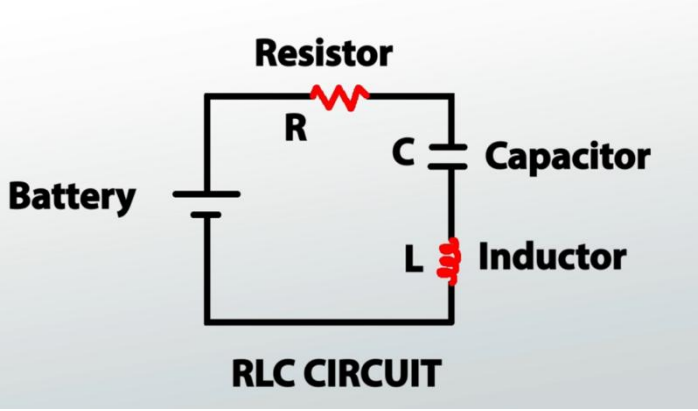


Figure a: A Standard RLC Circuit

To really understand why electrical devices don't just get to action when they are plugged in or turned on, we need to really understand what RLC (Resistor, Inductor, Capacitor) circuits are and the mathematics behind them. When we apply KVL to Kirchhoff's Voltage Law to a standard series RLC circuit as seen above, we uncover a second-order differential equation that forms the basis of any of our analysis in this project. The formula is listed below;

$$F(t) = L\frac{d^2q}{dt^2} + R\frac{dq}{dt} + \frac{1}{C}q(t)$$

The variables are defined below;

F(t) – The external voltage source applied to the circuit as a function of time

L – The inductance of the inductor

R- The resistance of the resistor

C – Capacitance of the capacitor

Q(t) – Electric charge stored on the capacitor at any time t

$\frac{dq}{dt}$ - Current flowing through the circuit

$\frac{d^2q}{dt^2}$ – Rate of change of current

This RLC formula is based on certain assumptions that make the circuit manageable whilst giving a realistic representation of the circuit behavior. We assume that the resistor only provides resistance R, the inductor only provides inductance L and the capacitor only stores charges with capacitance C. We also have to assume that at every instant, the voltage around a closed loop is 0, which enables us to write our equations. Also, we assume that the values of R, L and C do not change with time. As our analysis progresses, some of the assumptions used in solving and analyzing the circuit will be introduced and discussed.

As George Maria Selvam and Vignesh (2018) point out, we could simplify the circuit by looking at the interaction with just two rival forces, which are the damping coefficient and the natural resonant frequency.

The damping coefficient represents the system's internal friction, which is determined by the resistor, actively working to slow the current down. The natural resonant frequency represents the circuit's natural desire to vibrate and swing, dictated entirely by how the inductor and capacitor store and pass energy back and forth.

These terms are important because, depending on the relationship between the damping coefficient and the natural resonant frequency, the circuit may respond in three distinct ways: underdamped, critically damped, or overdamped. In an underdamped system, the resistance is too small to prevent the current from oscillating immediately, so the current and voltage rise and fall before settling to a steady value. This behaviour is common in radio tuning and communication systems, where these temporary oscillations help the circuit isolate and strengthen a particular signal or frequency. In a critically damped system, the circuit returns to equilibrium in the shortest possible time without oscillating, making it highly desirable in systems that require both speed and stability, such as precision measuring instruments and some control systems. In contrast, an overdamped system contains excessive damping, causing the response to return slowly to equilibrium without oscillating at all. Although slower, this behavior is useful in protective electrical systems and surge suppression circuits where preventing oscillations is more important than response speed. Together, these three

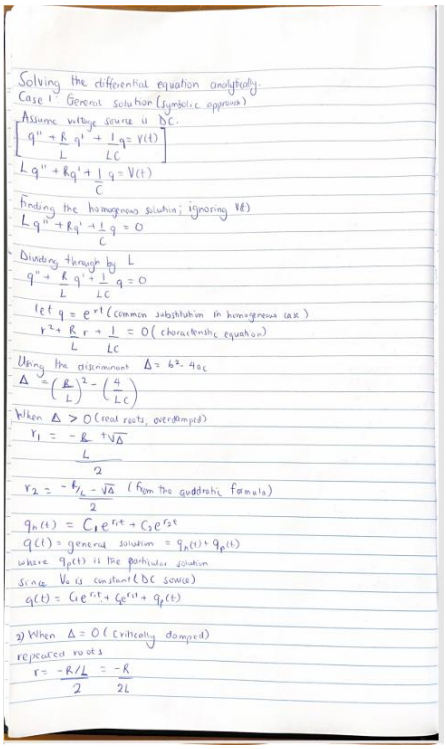
responses demonstrate how RLC circuits govern the stability of many practical engineering systems.

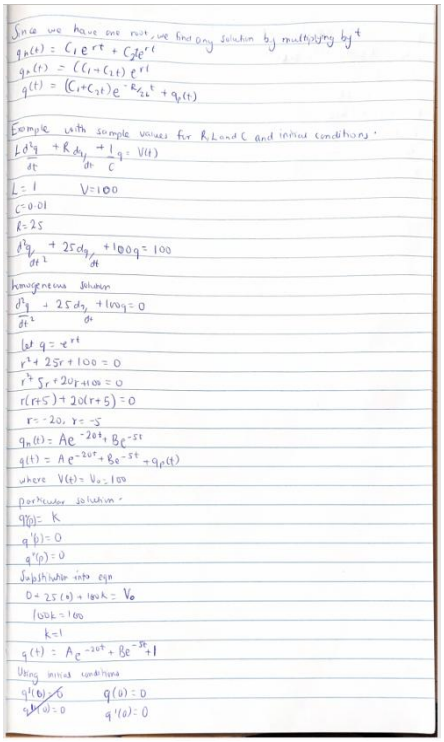
Without RLC dynamics, electrical systems would lose their ability to regulate how energy flows through a circuit over time. Signals and currents would change abruptly rather than gradually, leading to uncontrolled oscillations, overheating, signal distortion, and possible component failure. Devices such as communication systems, power supplies and filters rely heavily on the balance between resistance, capacitance and inductance of a circuit to maintain stability and safe operation. RLC dynamics, therefore, play a critical role not only in determining how circuits respond to disturbances but also in protecting modern electronic systems from instability and electrical damage.

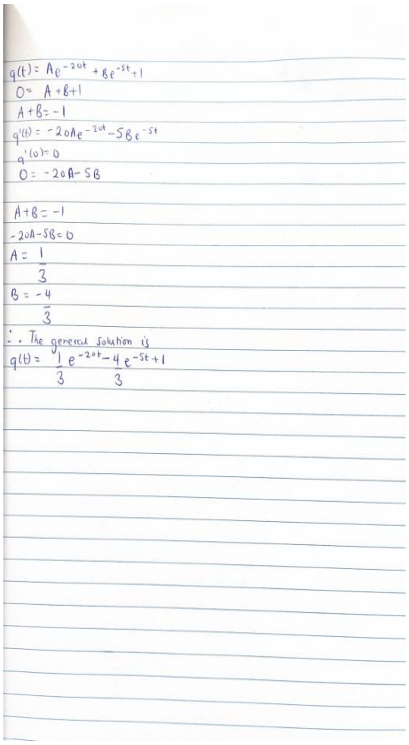
Analytical Solution Attempt

This initial attempt to solve the differential equation was based on the assumption that the voltage source is a direct current source and used initial conditions of q(0) = 0 and q`(0) = 0 (It was assumed that the charge at the moment the circuit was closed is 0). This analytical solution also does not contain a general solution for the differential equation when the system is underdamped. It contains solutions for when the system is overdamped or critically damped.

Solution







Initial Code Implementation and Basic Plots

As stated, RLC Series circuits can be analysed with respect to their behaviour under different damping conditions. A MATLAB simulation of such behaviours was performed using the initial code implementation of the base second-order differential equation in Appendix I. Conditions analysed included underdamping with an AC voltage source, as well as underdamping, critical damping and overdamping, all with a DC voltage source.

For the first case with an underdamped AC circuit (lines 9-27, Appendix I), an AC voltage of V = 100 sin (60) volts is applied across the circuit, and a low resistance of 2 ohms is used. The resulting plot in Figure (a) of Appendix II confirms that, due to this low resistance and the continuously changing AC, the current, or in this case, rate of charge oscillates for a very long time until settling, of which this time cannot be plotted due to its large value, The second case with an underdamped DC circuit with V=100V (lines 30- 48, Appendix I), shows a similar trend in Figure (b) of Appendix II. It can be noted that it does gradually settle. This is due to the fact that the constant voltage ensures that the transient term of the equation goes to zero, maintaining the steady state term.

The third case (lines 50-71, Appendix I) considers a critically damped RLC series circuit with a 20-ohm resistor. As seen from Figure (c) of Appendix II, the circuit reaches its steady state in the shortest possible time, as compared to the second case and the fourth case (lines 74-95, Appendix I), where a resistor of 100 ohms overdamps the circuit such that, though there no oscillations, it reaches a steady state gradually, as seen in Figure (d), Appendix II.

Completed Numerical Implementation

Three approaches were made to provide a numerical implementation for the ODE system. Their explanations and comparisons between them are as follows.

First is the Euler Method. The Euler method was implemented to numerically approximate the charge and current of the second-order RLC circuit by iteratively updating the state variables using a fixed step size, allowing comparison with more accurate numerical methods. The Euler method produced relatively larger numerical error when compared with ODE45 and RK4, showing its lower accuracy for this problem.

The second approach was RK4 (Runge–Kutta 4th Order). The fourth-order Runge–Kutta (RK4) method was used to solve the RLC circuit equations by evaluating intermediate slopes at each step, resulting in a highly accurate numerical approximation of charge and current. Comparison with ODE45 showed that RK4 produced extremely small errors, confirming its high accuracy and numerical stability.

Last of all was using ode45. MATLAB’s built-in ODE45 solver was used as a reference solution by solving the system of first-order differential equations with an adaptive step-size algorithm based on Runge–Kutta methods.

The implementation of each of these is provided in Appendix III.

Analytical Implementation

We made use of MATLAB's built-in function, dsolve, to solve our differential equation. We found the steady-state charge and current by finding the limit as our time approaches infinity. Our steady-state charge and current were 1 and 0, respectively.

In order to confirm the validity of our analysis (after obtaining our equation for Q), we substituted the first and second derivatives of Q into our original equation and compared the solution to the original differential equation's answer: Our test was successful.

Analytical-Numerical Comparison

After solving with both analytical and numerical methods, we decided to compare our results. We plotted graphs using our analytical method and displayed them side by side using the numerical methods(Appendix 4). The graphs were identical in each comparison. We went further and found the deviations our Numerical methods had from the Analytical solution, which we will discuss in our error section.

Solver Comparison

We then compared our numerical methods with MATLAB's ode45 built-in function. We plotted graphs for each numerical method (Euler and RK4) using ode45. Once again, our graphs appeared identical. To truly observe the discrepancies, we performed error analysis as shown below;

Error And Stability Analysis

Table *1*: Error Measurements Across Solvers

Solver	Average Current Error	Average Charge Error
Euler Method	1.0988×10^{-2}	1.4574×10^{-3}
Ode45	5.0919×10^{-5}	4.4732×10^{-6}
RK4	1.4940×10^{-7}	7.4000×10^{-9}

From the Euler Solver, the average current error is approximately 1.1×10^{-2}. Comparatively, the results are less accurate than those of the RK4 and the ode45. This is because the Euler Solver is a first-order method, which solves the differential equation by calculating the slope at every step. With a second-order term, it would have to move along a curved gradient and thus would end up overshooting, which accumulates the error over each iteration.

The RK4 solver produced error values which were very small, approximately 1.5×10^{-7} and 7.4×10^{-9}. This result is very close to the actual values since the error margins are very small and are attributed to the fact that the Runge-Kutta 4 method is a fourth-order method and can easily solve second-order terms within the differential equation.

The ode45 solver also had comparatively small error margins. MATLAB's built-in solver is adaptive and chooses huge step sizes when the curve is smooth and smaller step sizes when the curve gets sharp. This adaptive nature allows it to balance speed with accuracy and save memory.

From the errors observed as well, the RK4 is more accurate than the ode45 since the errors produced were 0.0000000074 and 0.0000044732, respectively. Since the initialisation of the step size was very small (h = 0.005), the RK4 solver produced minimal errors during computation, whereas ode45 optimises speed and accuracy when initialising the step and thus produces less accurate results compared to the RK4.

We also measured the errors between each numerical solver and ode45 and plotted using the semilogy function to observe extremely small numerical values, as shown in Appendix VI. The results from the graphs confirm the disparity found between the two numerical solvers and the analytical solver.

Parameter Sensitivity Analysis

Aside from damping effects, which can be realised by varying the system's resistance, varying other parameters, such as inductance and capacitance, also has effects.

Figure b in Appendix VII illustrates varying capacitance values in a critically damped example system, $\frac{d^2q}{dt^2} + 20\frac{dq}{dt} + \frac{q}{C} = 100$, for a 10-second time window. It can be seen that, increasing capacitance increases the time required for a system to reach steady state. This can be confirmed from the natural resonant frequency,

$$\omega_0 = \frac{1}{\sqrt{LC}}$$

such that increasing capacitance slows the rate at which energy swings back and forth, causing the system to take longer to reach a steady state.

Figure c) in Appendix VII also shows variations in inductance values and their effects on a critically damped system, $L\frac{d^2q}{dt^2} + 20\frac{dq}{dt} + \frac{q}{0.01} = 100$ for the same time window. It can be seen that, as inductance increases, the time required to reach steady state does change, but more notably, a higher inductance makes the system behave more underdamped, with the system oscillating more times before it reaches steady state. This can also be confirmed theoretically from the damping ratio,

$$\zeta = \frac{R}{R_c} = \frac{R}{\sqrt{4L/C}} = \frac{R}{2}\sqrt{\frac{C}{L}}$$

A damping ratio less than 1 indicates an underdamped system, equal to 1 indicates a critically damped system, and > 1 indicates an overdamped system. Hence, as L increases, the damping ratio starts to approach zero, giving it an underdamped behaviour and confirming the plotted results.

Conclusion

This project investigated the dynamic behaviour of a series RLC circuit through both analytical and numerical lenses. Starting from the second order differential equation obtained using Kirchhoff's law, the circuit response was examined under different damping conditions to understand how resistance, inductance and capacitance influence system behaviour. The analytical solution obtained using MATLAB's dsolve was successfully verified by substitution into the original differential equation and served as a benchmark for evaluating numerical methods.

Three numerical techniques were deployed in this project, namely: Euler, Runge-Kutta Fourth Order and MATLAB's ode45 solver. Although all three methods produced responses that closely matched the analytical solution, the error analysis showed significant differences in their accuracy. The Euler method accumulated the largest numerical whilst the RK4 produced the lowest errors. The ode45 also provided highly accurate results.

Also, the parameter sensitivity analysis further demonstrated that the transient response of an RLC circuit is strongly influenced by the values of resistance, inductance and capacitance. Increasing the capacitance increased the time required for the circuit to reach steady state, which, when increasing the inductance, reduced the damping ratio, causing the system to exhibit more oscillatory motions.

Overall, our project demonstrated how mathematical modelling paired with analytical solutions and numerical methods complement one another in analysing engineering systems. The results not only validated the implemented numerical method but also reinforced the importance of RLC Circuit dynamics in designing stable and reliable electrical systems

References

E. Ayankop-Andi, U. D. M., and G. J. Ibeh, “Effect of Transient and Frequency Response on RC and RLC Circuits to Instantaneous Forcing Function,” *Academy Journal of Science and Engineering*, vol. 11, no. 1, 2017.

George Maria Selvam, A., & Vignesh, D. (2018). *Dynamics in a Series RLC Circuit with Damping*. International Journal of Mathematics and Its Applications, 6(3), 1–6.

Appendices

Appendix I – Initial Code Implementation in MATLAB

```
% This file simulates multiple damping conditions with regard to the rate
% of change in charge with respect to time for RLC Series Circuits

% Declaration of sample values
L = 1; % in H
C = 0.01; % in F
R = 2; % in ohms (for underdamped)
```

```
%% Underdamped conditions with an AC source

syms q(t)
% from Lq'' + Rq' + q/c = V
% where V = 100 * sind(60*t) in V
[Vec_field] = odeToVectorField(diff(q, 2) == (100 * sin(60*t))/L - (q/C)/L - R*diff(q,1)/L);

odefun = matlabFunction(Vec_field, 'Vars',{'t', 'Y'});
tspan = [0,5];
q0 = [0; 0]; % q(0)=0, q'(0)=0

sol = ode45(odefun, tspan, q0);
figure(1)
fplot(@(x)deval(sol,x,1), tspan)
xlabel('t (sec)');
ylabel('q(t) (C)')
title('q(t) against from t=0 to t=0.5s for a Series RLC Circuit (Underdamped + AC Source)');
grid on
axis([-inf inf -0.75 0.75])
```

```
%% Underdamped conditions with a DC source

% from Lq'' + Rq' + q/c = V
% where V = 100 in V
syms q(t)
[Vec_field] = odeToVectorField(diff(q, 2) == (100)/L - (q/C)/L - R*diff(q,1)/L);

odefun = matlabFunction(Vec_field, 'Vars',{'t', 'Y'});
tspan = [0,7];
q0 = [0; 0]; % q(0)=0, q'(0)=0

sol = ode45(odefun, tspan, q0);
figure(2)
fplot(@(x)deval(sol,x,1), tspan)
xlabel('t (sec)');
ylabel('q(t) (C)')
title('q(t) against from t=0 to t=0.5s for a Series RLC Circuit (Underdamped + DC source)');
grid on
axis([-inf inf -0.25 2.5])
```

```
%% Critically damped condition with a DC source

% Declaration of sample values
R = 20; % in ohms

% from Lq'' + Rq' + q/c = V
% where V = 100 in V
syms q(t)
[Vec_field] = odeToVectorField(diff(q, 2) == (100)/L - (q/C)/L - R*diff(q,1)/L);

odefun = matlabFunction(Vec_field, 'Vars',{'t', 'Y'});
tspan = [0,5];
q0 = [0; 0];

sol = ode45(odefun, tspan, q0);
figure(3)
fplot(@(x)deval(sol,x,1), tspan)
xlabel('t (sec)');
ylabel('q(t) (C)')
title('q(t) against from t=0 to t=0.5s for a Series RLC Circuit (Critically damped + DC Source)');
grid on
axis([-inf inf -0.25 1.25])

```

```
%% Over damped condition with a DC source

% Declaration of sample values
R = 100; % in ohms

syms q(t)
% from Lq'' + Rq' + q/c = V
% where V = 100 in V
[Vec_field] = odeToVectorField(diff(q, 2) == (100)/L - (q/C)/L - R*diff(q,1)/L);

odefun = matlabFunction(Vec_field, 'Vars',{'t', 'Y'});
tspan = [0,10];
q0 = [0; 0];

sol = ode45(odefun, tspan, q0);
figure(4)
fplot(@(x)deval(sol,x,1), tspan)
xlabel('t (sec)');
ylabel('q(t) (C)')
title('q(t) against from t=0 to t=0.5s for a Series RLC Circuit (Over damped + DC source)');
grid on
axis([-inf inf -0.25 1.5])
```

Appendix II – Basic Plots (Solution vs Time plots)

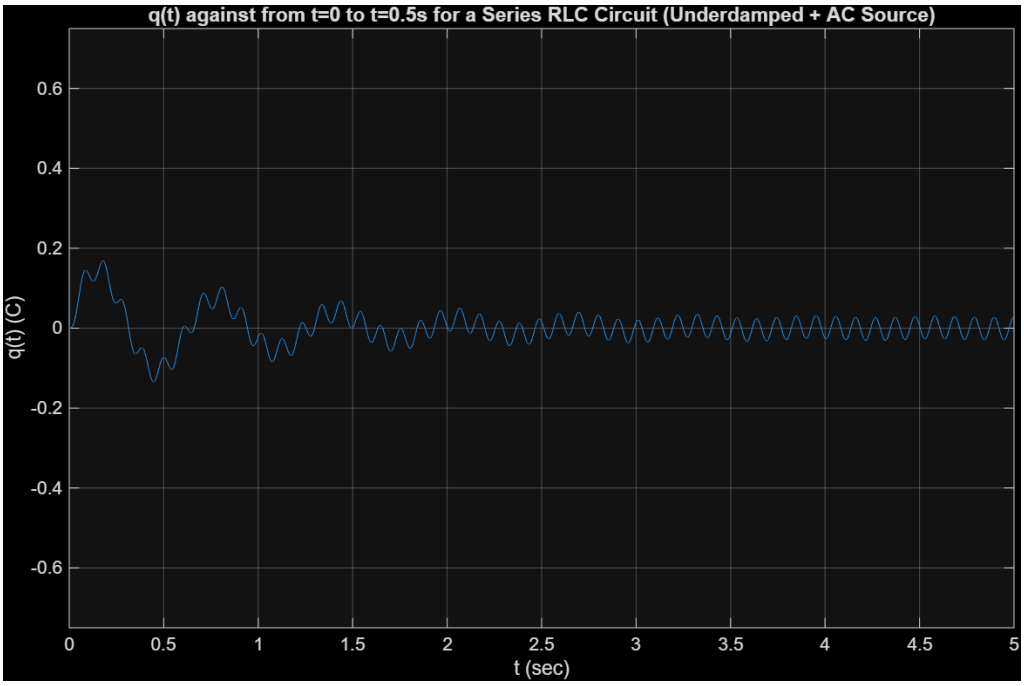


Figure a: Plot of q(t) changing with t for an AC source V = 100sin(60t) V and L = 1H, C = 0.01 F and R = 2 ohms (Underdamped)

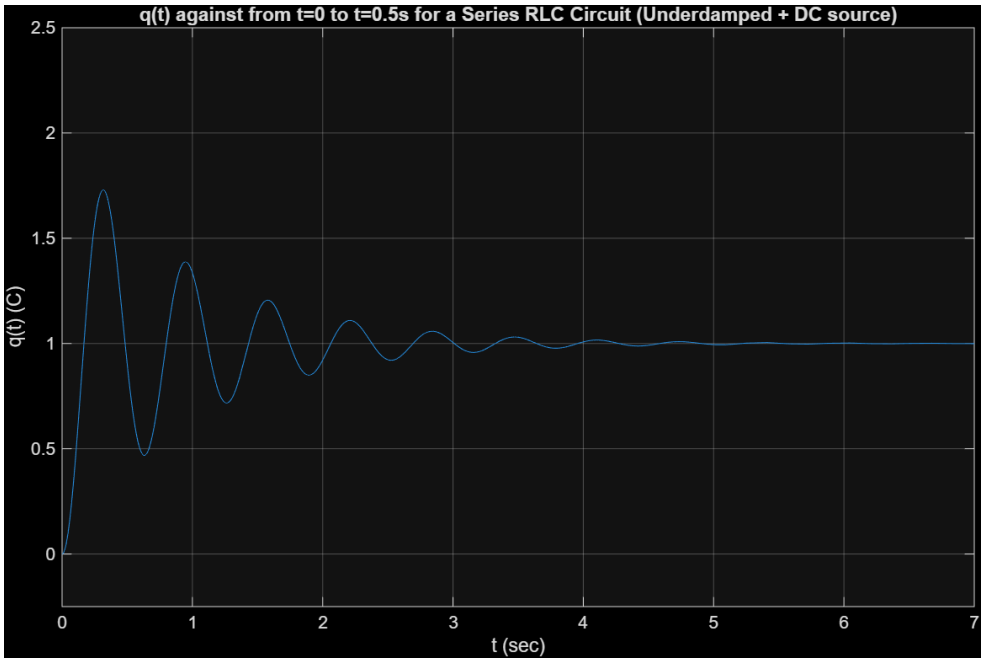


Figure b: Plot of q(t) changing with t for an DC source V = 100 V and L = 1H, C = 0.01 F and R = 2 ohms (Underdamped)

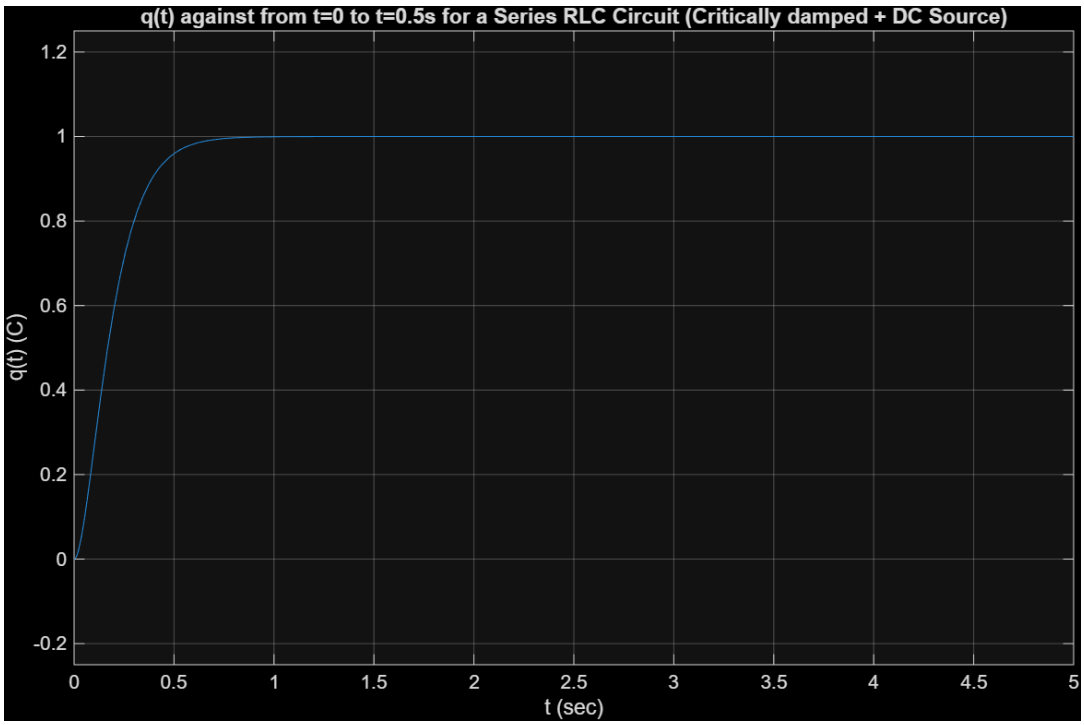


Figure c: Plot of q(t) changing with t for an DC source V = 100 V and L = 1H, C = 0.01 F and R = 20 ohms (Critically damped)

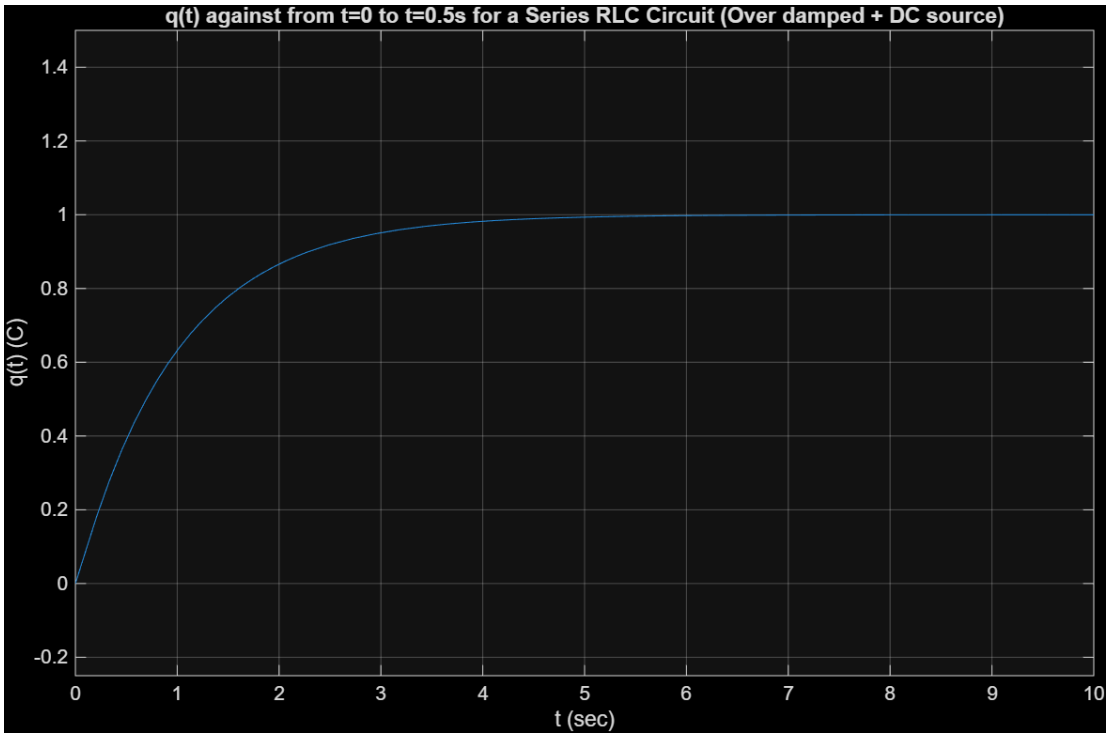


Figure d: Plot of q(t) changing with t for an DC source V = 100 V and L = 1H, C = 0.01 F and R = 100 ohms (Overdamped)

Appendix III – Completed Numerical Implementation

```
clc
clear
close all

% Circuit Parameters
L = 1;
R = 25;
C = 0.01;
V_source = 100;

% Initial conditions
q0 = 0;
i0 = 0;

%%  Solving using RK4
tic
h = 0.005;
t_rk4 = 0:h:2;
n = length(t_rk4);

q_rk4 = zeros(1, n);
i_rk4 = zeros(1, n);
q_rk4(1) = q0;
i_rk4(1) = i0;

for k = 1:n-1
    kq1 = i_rk4(k);
    ki1 = (V_source/L) - (R/L)*i_rk4(k) - (1/(L*C))*q_rk4(k);

    q_mid1 = q_rk4(k) + (h/2) * kq1;
    i_mid1 = i_rk4(k) + (h/2) * ki1;
    kq2 = i_mid1;
    ki2 = (V_source/L) - (R/L)*i_mid1 - (1/(L*C))*q_mid1;

    q_mid2 = q_rk4(k) + (h/2) * kq2;
    i_mid2 = i_rk4(k) + (h/2) * ki2;
    kq3 = i_mid2;
    ki3 = (V_source/L) - (R/L)*i_mid2 - (1/(L*C))*q_mid2;

    q_end = q_rk4(k) + h * kq3;
    i_end = i_rk4(k) + h * ki3;
    kq4 = i_end;
    ki4 = (V_source/L) - (R/L)*i_end - (1/(L*C))*q_end;

    q_slope = (kq1 + 2*kq2 + 2*kq3 + kq4) / 6;
    i_slope = (ki1 + 2*ki2 + 2*ki3 + ki4) / 6;

    q_rk4(k+1) = q_rk4(k) + h * q_slope;
    i_rk4(k+1) = i_rk4(k) + h * i_slope;
end
disp('RK4')
toc
```

Figure a: Numerical implementation using the Runge-Kutta 4th Order Method

```
%% Solving using ODE45
tic
y0 = [q0; i0];

[t_ode, y_ode] = ode45(@(t, y) [ ...
    y(2); ...
    (V_source/L) - (R/L)*y(2) - (1/(L*C))*y(1) ...
], t_rk4, y0);

q_ode = y_ode(:,1);
i_ode = y_ode(:,2);

disp('ODE45')
toc
```

Figure b: Numerical implementation using the MATLAB built-in solver, ode45

```
%% Solving using euler

tic
% Time interval
t_eu = t_rk4; %using the rk4 time so that direct comparison can be made later
n = length(t_eu);

% Initial conditions
q_eu = zeros(1,n);      % charge
i_eu = zeros(1,n);      % current = dq/dt

q_eu(1) = 0;
i_eu(1) = 0;

% Euler method loop
for k = 1:n-1

    dqdt = i_eu(k);
    didt = 100 - 25*i_eu(k) - 100*q_eu(k);

    q_eu(k+1) = q_eu(k) + h*dqdt;
    i_eu(k+1) = i_eu(k) + h*didt;

end
disp('Euler')
toc
```

Figure c: Numerical implementation using the Euler Method

Appendix IV – Analytical-Numerical Comparison Plots

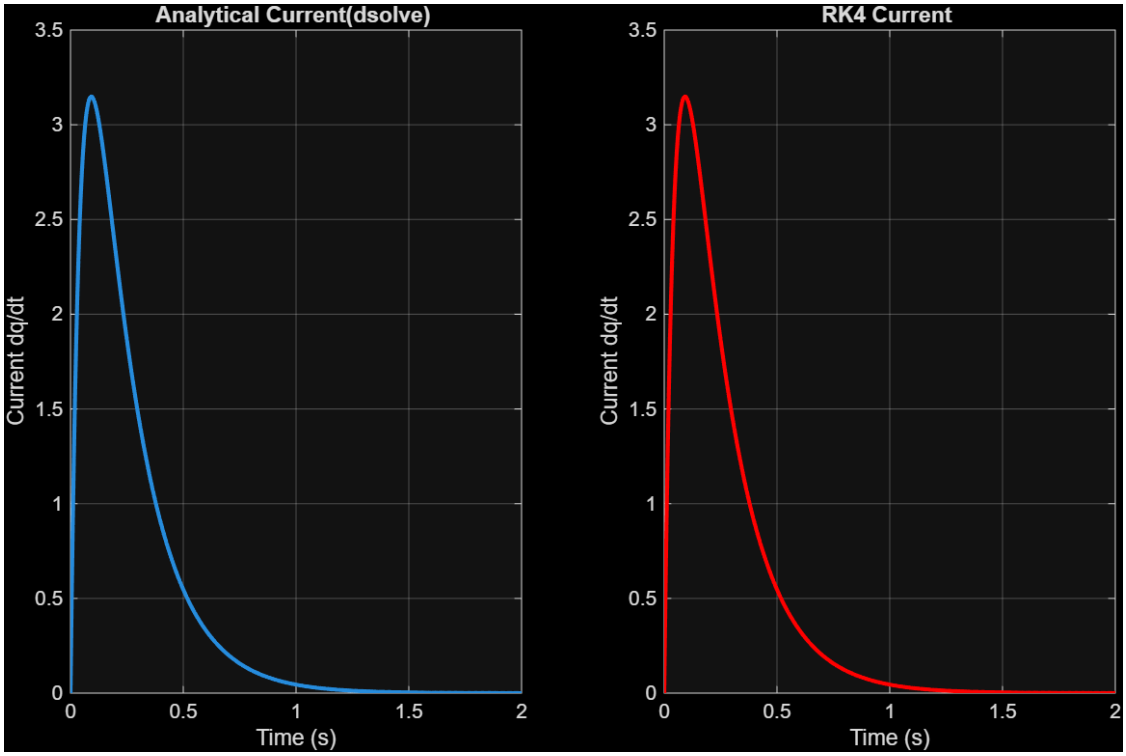


Figure a: Current Comparison of RK4 (Right) and dsolve (Left)

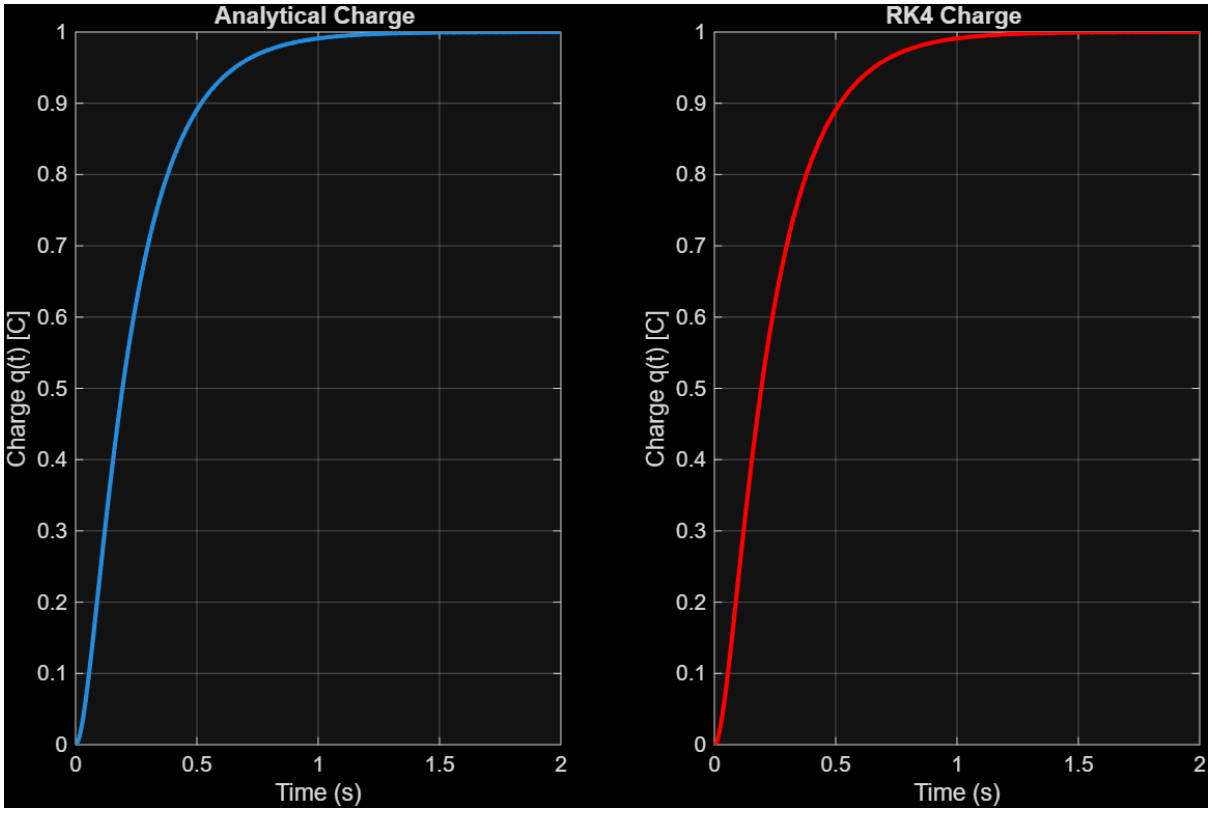


Figure b: Charge Comparison of RK4 (Right) and dsolve (Left)

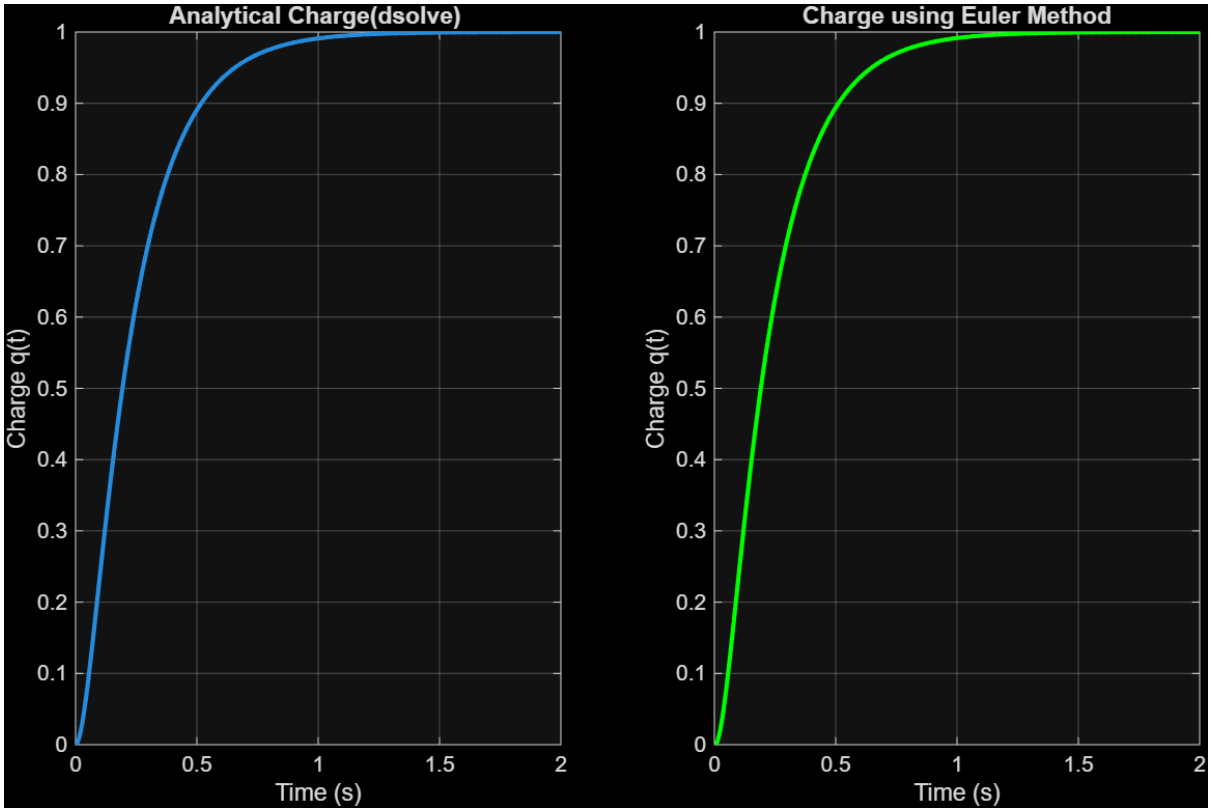


Figure c: Charge Comparison of the Euler Method (Right) and dsolve (Left)

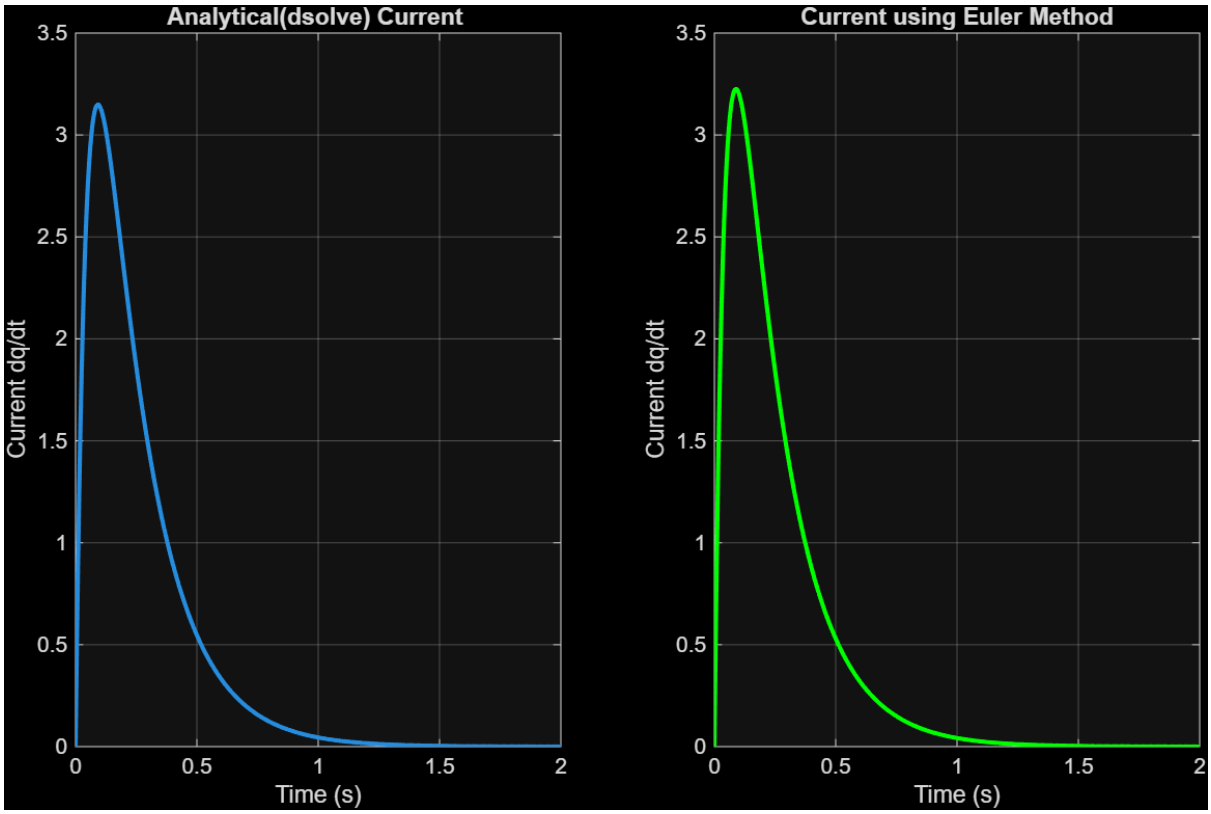


Figure d: Current Comparison of the Euler method and dsolve

Appendix V – Solver Comparison Plots

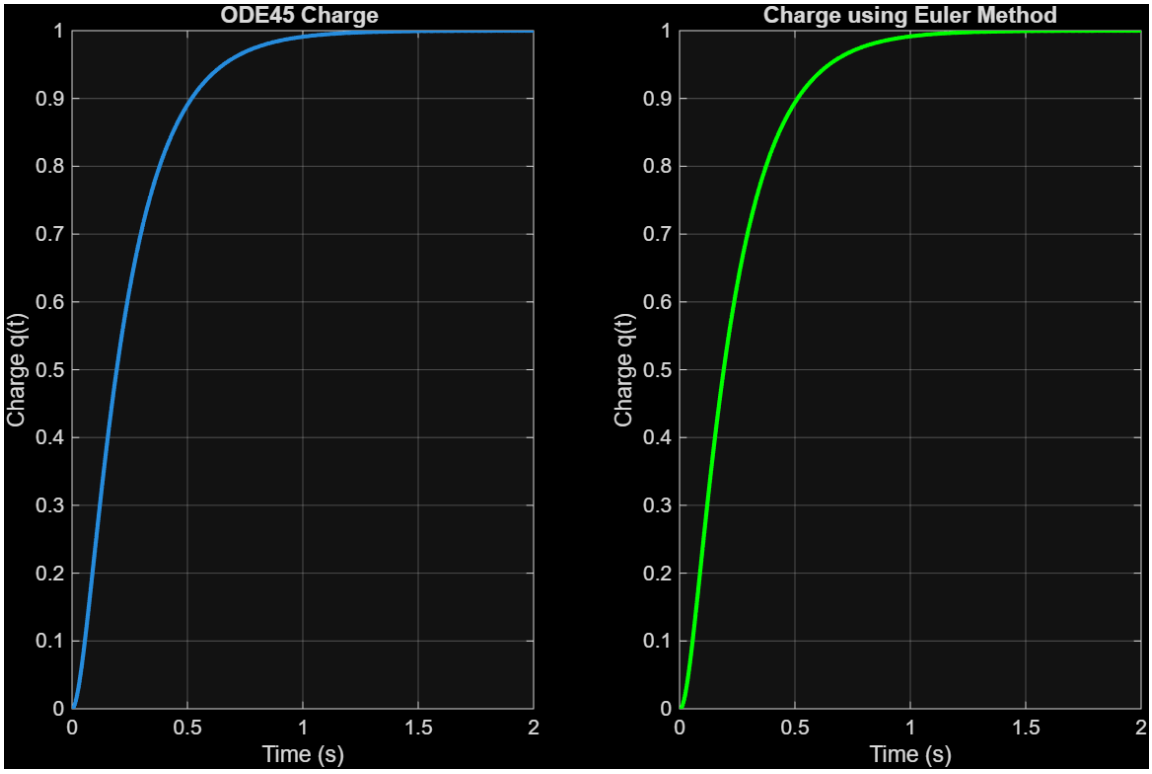


Figure a: Charge Comparison between using the numerical Euler method (Right), and MATLAB's ode45 solver (Left)

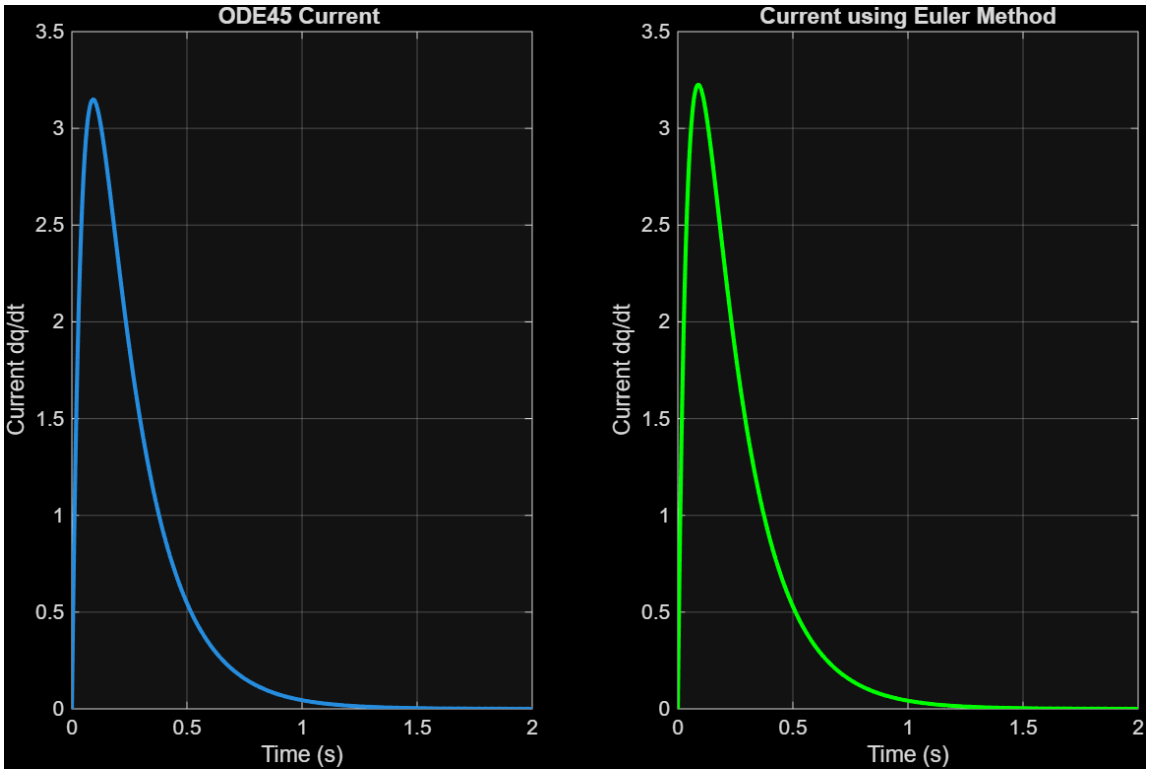


Figure b: Current Comparison between using the numerical Euler method (Right), and MATLAB's ode45 solver (Left)

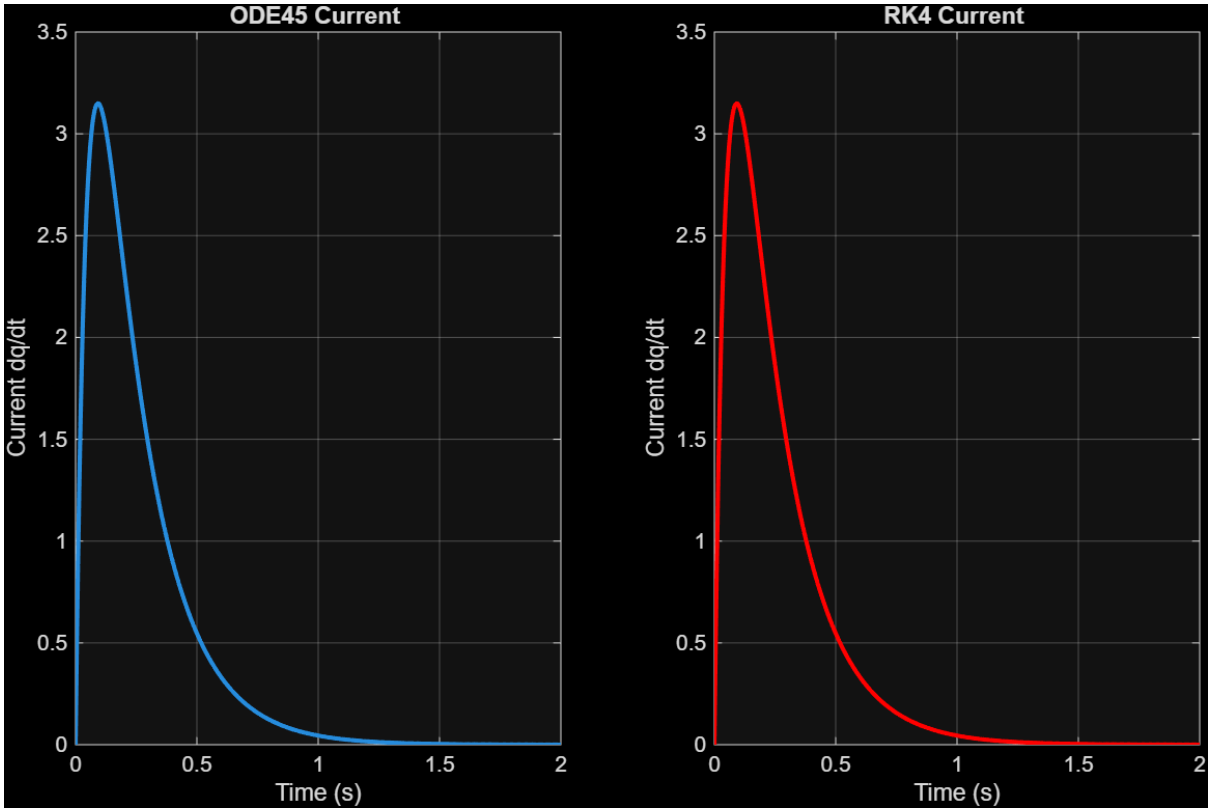


Figure c: Charge Comparison between using the numerical RK4 method (Right), and MATLAB's ode45 solver (Left)

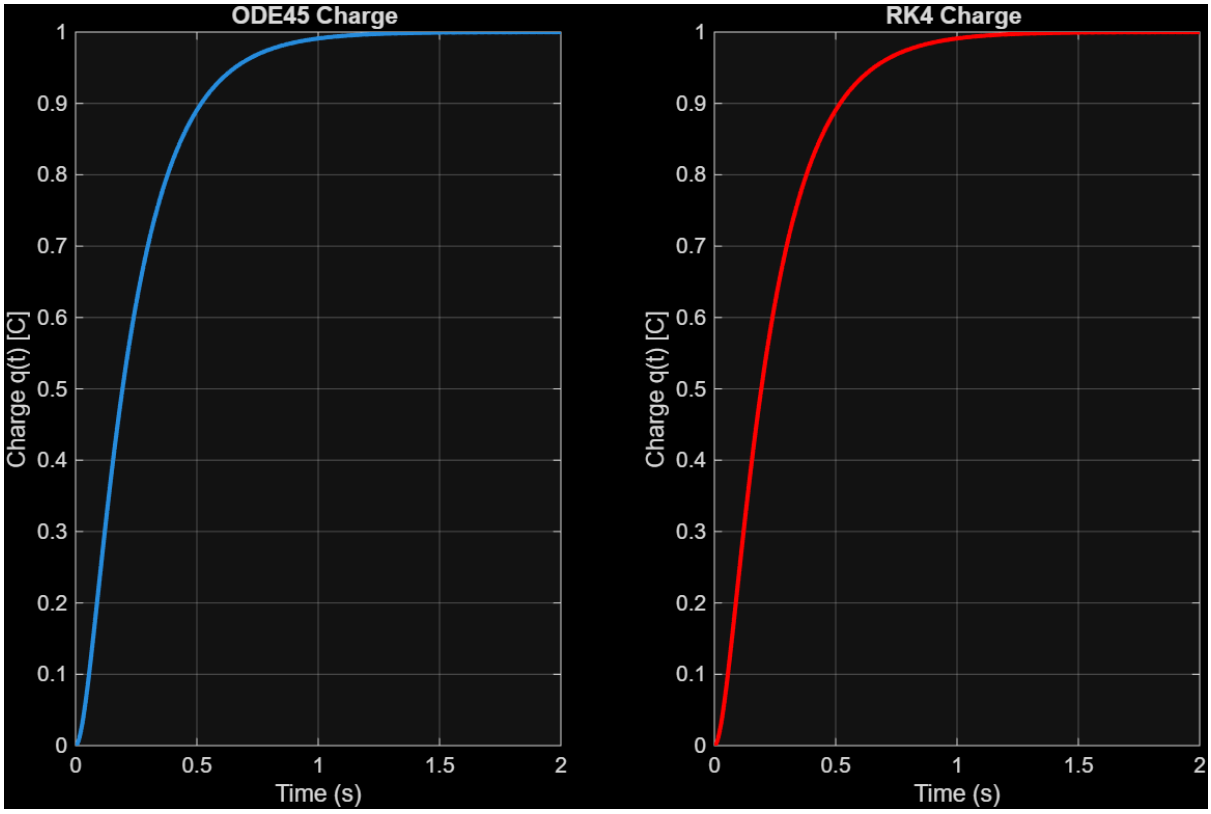


Figure d: Charge Comparison between using the numerical RK4 method (Right), and MATLAB's ode45 solver (Left)

```
93   %%
94   % Plot charge
95   figure
96   subplot(1, 2, 1)
97   plot(t_ode, q_ode,'LineWidth',2)
98   grid on
99   xlabel('Time (s)')
100  ylabel('Charge q(t)')
101  title('ODE45 Charge ')
102
103  subplot(1,2,2)
104  plot(t_eu,q_eu,'LineWidth',2,color="green")
105  grid on
106  xlabel('Time (s)')
107  ylabel('Charge q(t)')
108  title('Charge using Euler Method')
109
110  % Plot current
111  % ODE45 Current
112  figure
113  subplot(1, 2, 1)
114  plot(t_ode, i_ode, 'LineWidth', 2)
115  grid on
116  xlabel('Time (s)')
117  ylabel('Current dq/dt')
118  title('ODE45 Current ')
119
120  %Euler current
121  subplot(1,2,2)
122  plot(t_eu,i_eu,'LineWidth',2,color="green")
123  grid on
124  xlabel('Time (s)')
125  ylabel('Current dq/dt')
126  title('Current using Euler Method')
127
```

Figure d: Code implementation for Figures a and b

```
%% Comparing rk4

% ODE45 Current
figure
subplot(1, 2, 1)
plot(t_ode, i_ode,  'LineWidth', 2)
grid on
xlabel('Time (s)')
ylabel('Current dq/dt')
title('ODE45 Current ')

% RK4 Current
subplot(1, 2, 2)
plot(t_rk4, i_rk4, 'LineWidth', 2,color="red")
grid on
xlabel('Time (s)')
ylabel('Current dq/dt')
title('RK4 Current ')

```

```
%% Comparing rk4

% Left: ODE45 Charge
figure
subplot(1, 2, 1)
plot(t_ode, q_ode,'LineWidth', 2)
grid on
xlabel('Time (s)')
ylabel('Charge q(t) [C]')
title('ODE45 Charge ')

% Right: RK4 Charge
subplot(1, 2, 2)
plot(t_rk4, q_rk4, 'LineWidth', 2,color="red")
grid on
xlabel('Time (s)')
ylabel('Charge q(t) [C]')
title('RK4 Charge ')
```

Figure d: Code implementation for Figures c and d

Appendix VI – Error plots

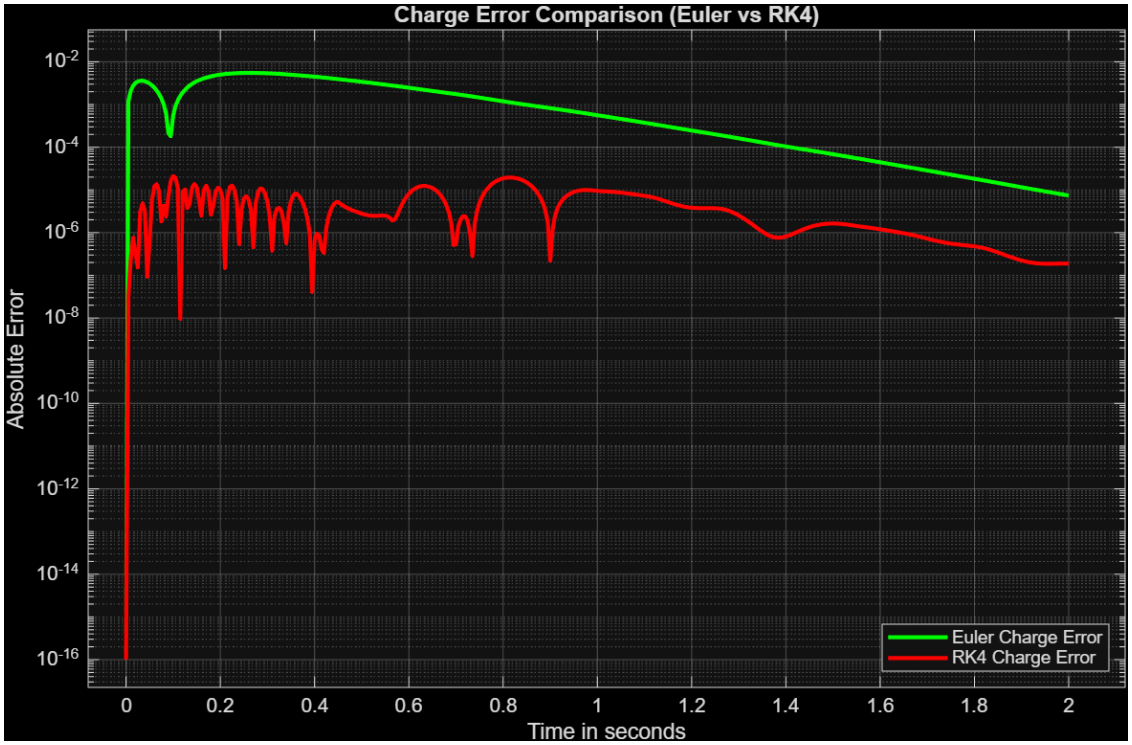


Figure a: Charge Error Profile, comparing Euler Method and RK4 Method

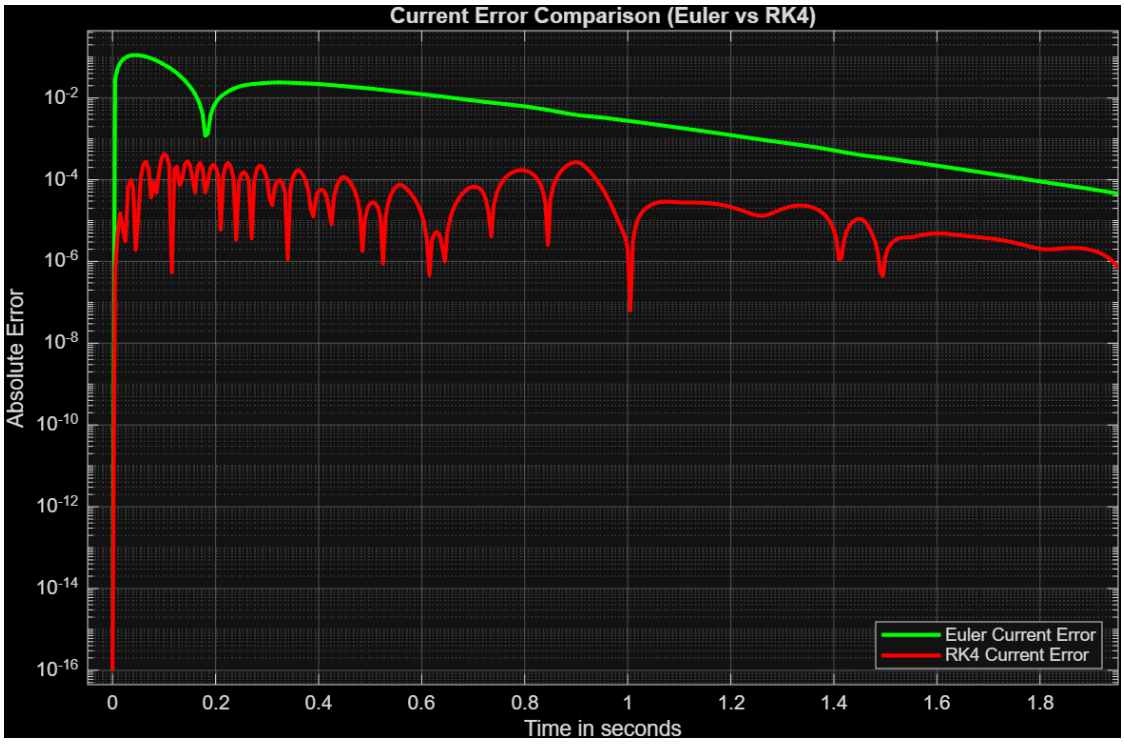


Figure b: Current Error Profile, comparing Euler Method and RK4 Method

```
%% Error Comparison Diagrams

% Euler Method Error Profile
figure()
semilogy(t_eu, q_error_eu + 1e-16, 'g-', 'LineWidth', 2);
hold on;
semilogy(t_rk4, q_error_rk + 1e-16, 'r-', 'LineWidth', 2);
grid on;
xlabel('Time in seconds')
ylabel('Absolute Error')
title('Charge Error Comparison (Euler vs RK4)')
legend('Euler Charge Error', 'RK4 Charge Error', 'Location', 'SouthEast');

% Rk4 Error Profile
figure()
semilogy(t_eu, i_error_eu + 1e-16, 'g-', 'LineWidth', 2);
hold on;
semilogy(t_rk4, i_error_rk + 1e-16, 'r-', 'LineWidth', 2);
grid on;
xlabel('Time in seconds')
ylabel('Absolute Error')
title('Current Error Comparison (Euler vs RK4)')
legend('Euler Current Error', 'RK4 Current Error', 'Location', 'SouthEast');
```

Figure c: Code implementation for both charge and current error profiles

```
%% Solving for errors
% Checking rk4
% ode45 outputs column vectors, so we flip them into row vectors
q_ode = y_ode(:,1)';
i_ode = y_ode(:,2)';

q_error_rk = abs(q_rk4 - q_ode);
i_error_rk = abs(i_rk4 - i_ode);

sum_Qrk_errors = sum(q_error_rk);
avg_Qrk_error = sum_Qrk_errors / length(q_error_rk);

sum_Irk_errors=sum(i_error_rk);
avg_Irk_error=sum_Irk_errors/length(i_error_rk);

fprintf("\nError when using rk4 as compared to ode45.\nCurrent : %.10f\nCharge : %.10f\n ",avg_Irk_error,avg_Qrk_error);


% Checking euler

q_error_eu = abs(q_eu - q_ode);
i_error_eu = abs(i_eu - i_ode);

sum_Qeu_errors = sum(q_error_eu);
avg_Qeu_error = sum_Qeu_errors / length(q_error_eu);

sum_Ieu_errors=sum(i_error_eu);
avg_Ieu_error =sum_Ieu_errors/length(i_error_eu);

fprintf(" \nError when using Euler as compared to ode45.\nCurrent : %.10f\nCharge : %.10f ",avg_Ieu_error,avg_Qeu_error);
```

Figure d: Code implementation for obtaining error values for RK4 and Euler methods, taking the ODE45 as the baseline

Appendix VII – Parameter Sensitivity Plots

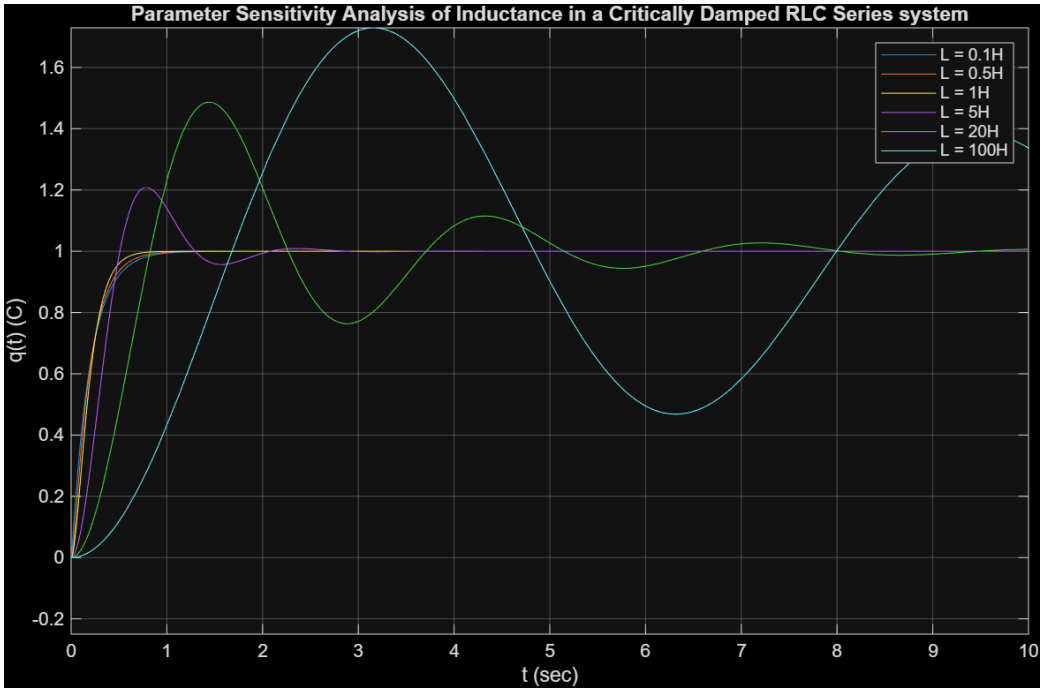


Figure a: Parameter Sensitivity Analysis of Inductance in a Critically Damped RLC Series system

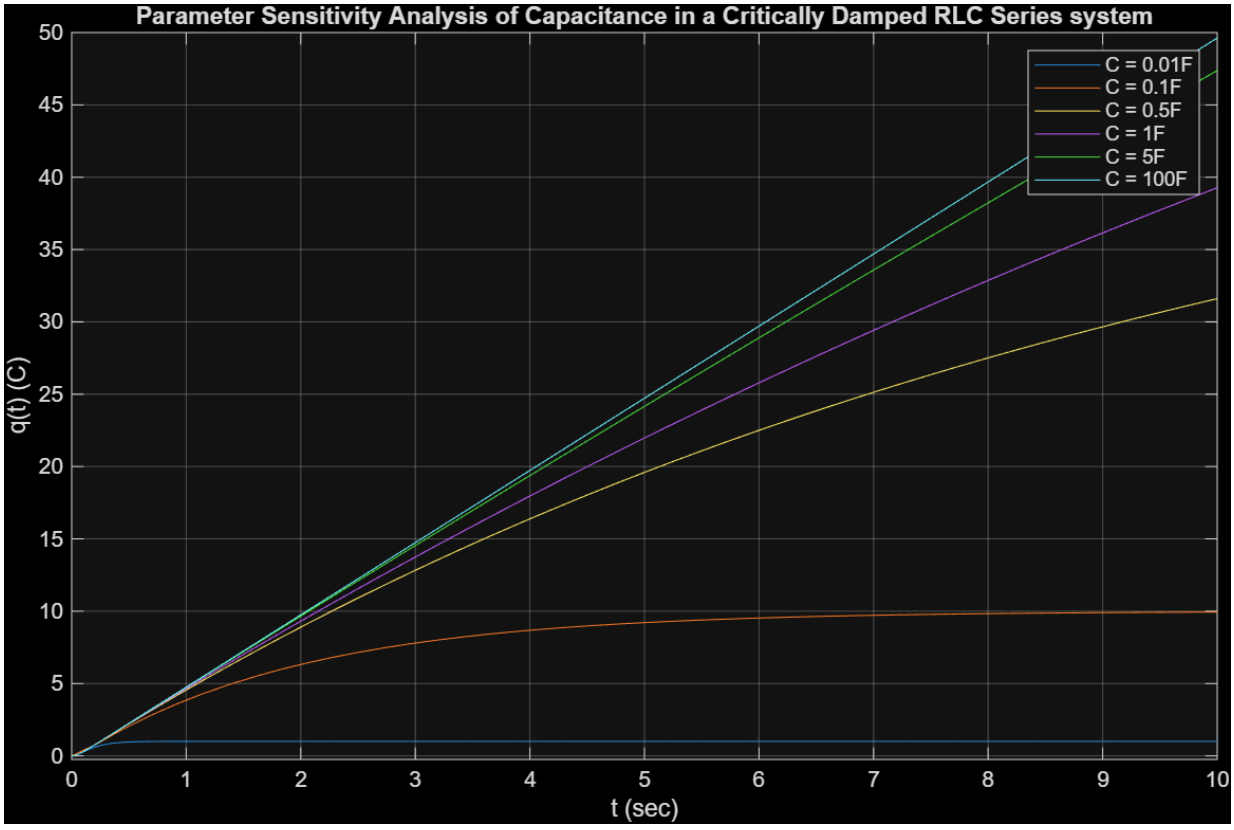


Figure b: Parameter Sensitivity Analysis of Capacitance in a Critically Damped RLC Series system

Appendix VIII – Output of Code Implementation

Calculating analytical solution with dsolve:

Charge Equation q(t):
exp(-20*t_time)/3 - (4*exp(-5*t_time))/3 + 1

Current Equation i(t):
(20*exp(-5*t_time))/3 - (20*exp(-20*t_time))/3

Equilibrium Charge: 1.0000 C
Cleaned Up Charge Expression q(t):
exp(-20*t_time)/3 - (4*exp(-5*t_time))/3 + 1

Cleaned Up Current Expression i(t):

(20*exp(-20*t_time)*(exp(15*t_time) - 1))/3

Analytical solution mathematically verified.

Error when using rk4 as compared to the exact analytical solution.
Current : 0.0000001494
Charge : 0.0000000074

Error when using Euler as compared to exact analytical solution.
Current : 0.0109882022
Charge : 0.0014574129
Error when using ode45 as compared to exact analytical solution.
Current : 0.0000509193
Charge : 0.0000044732
Error when using rk4 as compared to ode45.
Current : 0.0000509829
Charge : 0.0000044763

Error when using Euler as compared to ode45.
Current : 0.0109771345
Charge : 0.0014571219

Appendix IX – Code Implementation for Analytical Solution

```
%%  Solving analytically using dsolve
fprintf('Calculating analytical solution with dsolve:\n');


syms t_time q_sym(t_time) q
ode_eq = 1*diff(q_sym, t_time, 2) + 25*diff(q_sym, t_time, 1) + 100*q_sym == 100;

Dq=diff(q_sym,t_time,1);
conds = [q_sym(0) == q0, Dq(0) == i0];

q_expr = dsolve(ode_eq, conds);
i_expr = diff(q_expr, t_time);

fprintf('\n Charge Equation q(t):\n');
disp(q_expr);
fprintf('Current Equation i(t):\n');
disp(i_expr);

% Evaluating the math over our  numerical time grid points
q_exact = double(subs(q_expr, t_time, t_rk4));
i_exact = double(subs(i_expr, t_time, t_rk4));

% Finding steady-state charge
q_steady = limit(q_exact, t_time, inf);

% Finding steady-state current
i_steady = limit(i_exact, t_time, inf);

fprintf('Steady-State Charge: %.4f C\n', q_steady);
fprintf('Steady-State Current: %.4f A\n', i_steady);

%finding equilibrium by equating derivatives to zero
eq_eq = subs(ode_eq, [diff(q_sym, t_time, 2), diff(q_sym, t_time, 1),q_sym], [0, 0,q]);
```

```
q_equilibrium = solve(eq_eq, q);

fprintf('Equilibrium Charge: %.4f C\n', q_equilibrium);

%Symbolic confirmation and verification
q_simplified = simplify(q_expr);
i_simplified = simplify(i_expr);

fprintf('Cleaned Up Charge Expression q(t):\n');
disp(q_simplified);
fprintf('Cleaned Up Current Expression i(t):\n');
disp(i_simplified);


%if our differential solution is correct then lhs=rhs
lhs_side = 1*diff(q_simplified, t_time, 2) + 25*diff(q_simplified, t_time, 1) + 100*q_simplified;
rhs_side = 100;



zero_check = simplify(lhs_side - rhs_side);

if zero_check == 0
    fprintf('Analytical solution mathematically verified.\n');
else
    fprintf('Analytical Solution is wrong!!!!!!!!!!!!!!!\n');
end
```